A MINI PROJECT REPORT

ON

UNDO AND REDO TEXT EDITOR

(Using Stack Data Structure in C++)

Submitted in partial fulfilment of the requirements

for the award of the degree of

BACHELOR OF COMPUTER APPLICATIONS / ENGINEERING

Submitted By:

Akhil Pratap Singh

[12410802]

Under the Guidance of:

Puneet Gupta

[Department of Computer Science AI/ML]

Lovely professional university

CERTIFICATE

This is to certify that the mini project report entitled “Undo and Redo Text Editor” is a bona fide work carried out by Akhil Pratap Singh, in partial fulfilment of the requirements for the award of the degree in Computer Science / Computer Applications during the academic year 2025-26.

The project has been carried out under my supervision and guidance. The work presented in this report is original and has not been submitted elsewhere for the award of any other degree or diploma.

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my guide, for the valuable guidance, encouragement, and support provided throughout the development of this project, “Undo and Redo Text Editor”.

I am also thankful to the Head of the Department and all the faculty members of the for providing the necessary resources and a suitable environment to complete this project successfully.

Finally, I extend my heartfelt thanks to my family and friends for their constant motivation and support during the course of this project.

Akhil Pratap Singh

ABSTRACT

Most text editors allow a user to undo a mistake and redo a change that was undone. This project, “Undo and Redo Text Editor”, is a small console-based application written in C++ that demonstrates how this common feature can be implemented using a simple and well-known data structure called the Stack.

The editor allows the user to type text, view the current text, undo the last change, redo an undone change, clear the text, and exit the program. Two stacks are used internally: one to store the history of previous states (for undo) and another to store states that were undone (for redo). Whenever the user makes a change, the previous state of the text is pushed onto the undo stack. When the user chooses undo, the top of the undo stack is restored and the current state is pushed onto the redo stack, and vice-versa for redo.

This project is intended purely for academic learning. It focuses on understanding the working of the Stack data structure (Last In First Out - LIFO) through a practical and easy-to-follow example, rather than building a full-featured or production-level text editor.

1. Introduction

A text editor is a program that allows users to type, edit, and manage text. While using any editor, mistakes are common - a user may type something incorrectly or delete something by accident. To handle such situations, most modern editors provide an Undo option to reverse the last action, and a Redo option to re-apply an action that was undone.

This project demonstrates a simplified version of this feature using the C++ programming language and the Stack data structure available in the Standard Template Library (STL). The project is console based (text based, without any graphical interface) so that the focus remains completely on the logic of undo and redo rather than on visual design.

2. Objective

- To understand and apply the Stack data structure in a real, practical example.
- To design and implement a simple console-based text editor in C++.
- To implement the Undo and Redo operations using two separate stacks.
- To strengthen the understanding of Last In First Out (LIFO) behaviour.
- To practice writing modular, readable, and well-commented C++ code.

3. Problem Statement

Beginner programmers often study the Stack data structure only in theory, without seeing a real application of it. There is a need for a small, easy-to-understand project that clearly shows how a stack can be used to solve a real everyday problem - in this case, remembering previous versions of a piece of text so that changes can be undone and redone. This project addresses that need by building a minimal text editor whose only purpose is to demonstrate the undo/redo mechanism clearly.

4. Data Structure Used: Stack

A Stack is a linear data structure that follows the Last In First Out (LIFO) principle. This means that the last element inserted (pushed) into the stack is the first element to be removed (popped) from it. The two basic operations on a stack are:

- Push - insert an element on top of the stack.
- Pop - remove the element from the top of the stack.
- Top / Peek - look at the top element without removing it.

In this project, the C++ Standard Template Library (STL) container std::stack<string> is used. Two separate stacks are maintained:

- undoStack - stores every previous version of the text before a change is made.
- redoStack - stores a version of the text right after it has been undone.

This exactly matches the LIFO nature of undo/redo: the most recent change is always the first one to be undone, and the most recently undone change is always the first one to be redone.

5. Algorithm

5.1 Main Program Algorithm

1. Start the program.
2. Initialize an empty string currentText, an empty undoStack, and an empty redoStack.
3. Display the menu with the options: Enter Text, Display Text, Undo, Redo, Clear Text, Exit.
4. Read the user's choice.
5. If choice = 1: Read new text from the user and call the Add Text algorithm.
6. If choice = 2: Display the current text to the user.
7. If choice = 3: Call the Undo algorithm.
8. If choice = 4: Call the Redo algorithm.
9. If choice = 5: Call the Clear Text algorithm.
10. If choice = 6: Print a goodbye message and stop the program.
11. If the choice is invalid, show an error message.
12. Repeat steps 3 to 11 until the user selects Exit.

5.2 Add Text Algorithm

1. Push the current text onto the undoStack (save old state).
2. Append the new text entered by the user to the current text.
3. Empty the redoStack, since a new change makes old redo history invalid.

5.3 Undo Algorithm

1. If the undoStack is empty, display “Nothing to undo” and stop.
2. Otherwise, push the current text onto the redoStack.
3. Pop the top value from the undoStack and make it the current text.

5.4 Redo Algorithm

1. If the redoStack is empty, display “Nothing to redo” and stop.

2. Otherwise, push the current text onto the undoStack.

3. Pop the top value from the redoStack and make it the current text.

5.5 Clear Text Algorithm

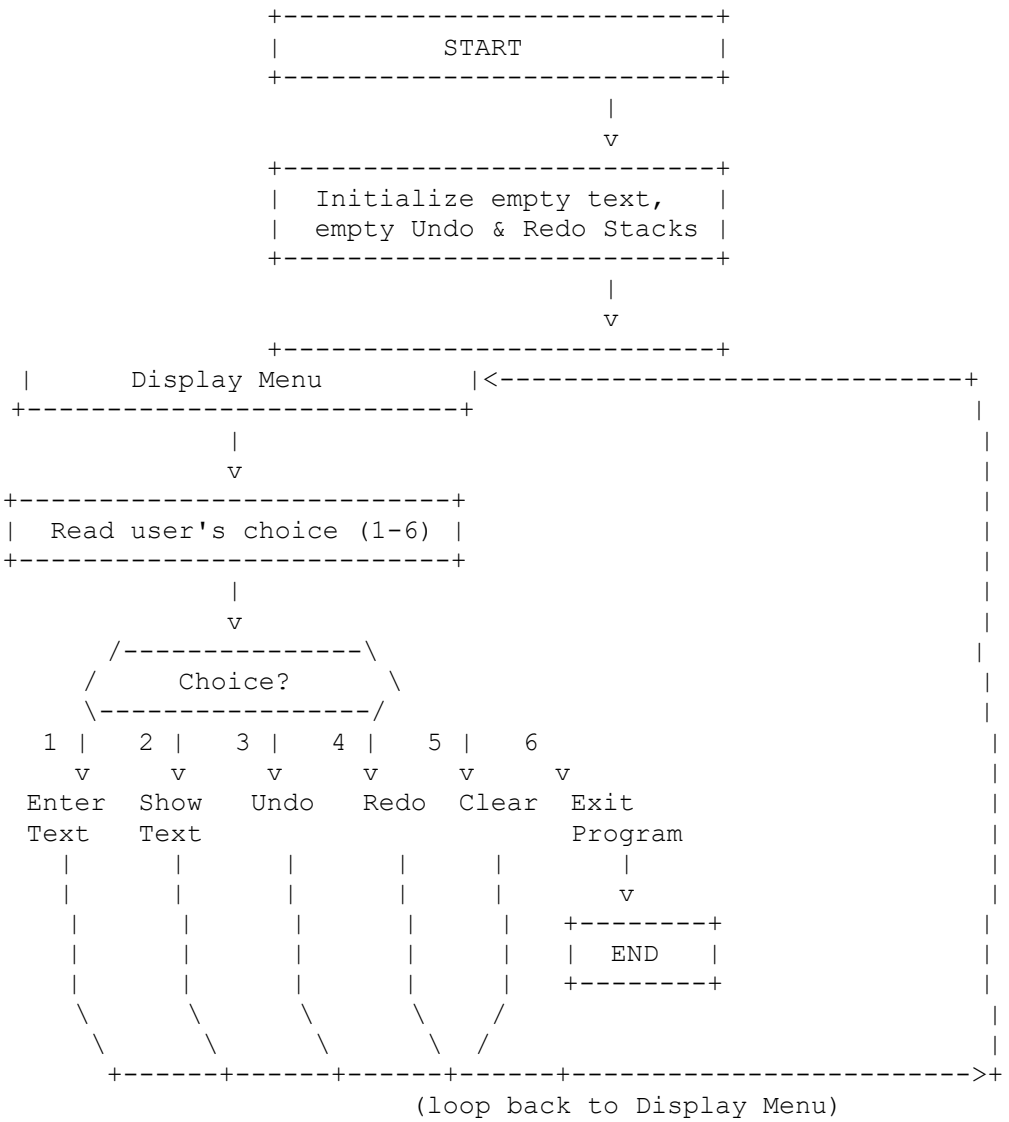
1. Push the current text onto the undoStack (so clearing can also be undone).

2. Set the current text to an empty string.

3. Empty the redoStack.

6. Flowchart

The flowchart below shows the overall flow of the program in a simple, text (ASCII) format:



7. Program Explanation

The program is organized around a single class named TextEditor. This class keeps the current text in a string variable and keeps the two stacks (undoStack and redoStack) as its private data members. All operations of the editor are implemented as member functions of this class, which keeps the code modular and easy to follow.

- addText() - saves the old text on undoStack, then appends the new text to the current text, and clears redoStack.
- clearText() - saves the old text on undoStack, then empties the current text.
- undo() - moves the current text to redoStack and restores the last saved text from undoStack.
- redo() - moves the current text to undoStack and restores the last undone text from redoStack.
- display() - simply prints the current text, or a message if it is empty.

The main() function contains a menu-driven loop using a do-while loop and a switch statement. It repeatedly shows the menu, reads the user's choice, and calls the matching function of the TextEditor object, until the user selects the Exit option. Basic input validation is also added so that the program does not crash if the user types a letter instead of a number.

8. Complete Source Code

File name: undo_redo_editor.cpp

```cpp
#include <iostream>
#include <stack>
#include <string>
#include <limits>

using namespace std;

class TextEditor
{
private:
    string currentText;
    stack<string> undoStack;
    stack<string> redoStack;

public:

    TextEditor()
    {
        currentText = "";
    }


    void addText(const string &newPart)
    {
        undoStack.push(currentText);
        currentText += newPart;


        while (!redoStack.empty())
            redoStack.pop();

        cout << "Text added successfully.\n";
    }


    void clearText()
    {
        undoStack.push(currentText);
        currentText = "";
```

```
void clearText()
    while (!redoStack.empty())
        redoStack.pop();

    cout << "Text cleared.\n";
}


void undo()
{
    if (undoStack.empty())
    {
        cout << "Nothing to undo!\n";
        return;
    }

    redoStack.push(currentText);
    currentText = undoStack.top();
    undoStack.pop();

    cout << "Undo successful.\n";
}


void redo()
{
    if (redoStack.empty())
    {
        cout << "Nothing to redo!\n";
        return;
    }

    undoStack.push(currentText);
    currentText = redoStack.top();
    redoStack.pop();

    cout << "Redo successful.\n";
}
```

```
    void display() const
    {

        if (currentText.empty())
            cout << "[Text box is empty]\n";
        else
            cout << "Current Text: " << currentText << "\n";

    }
};

void showMenu()
{
    cout << "\n TEXT EDITOR MENU \n";
    cout << "1. Enter Text\n";
    cout << "2. Display Current Text\n";
    cout << "3. Undo\n";
    cout << "4. Redo\n";
    cout << "5. Clear Text\n";
    cout << "6. Exit\n";
    cout << "Enter your choice (1-6): ";
}


int main()
{
    TextEditor editor;
    int choice;

    cout << "***** WELCOME TO UNDO/REDO TEXT EDITOR *****\n";

    do
    {
        showMenu();
        cin >> choice;


        if (cin.fail())
```

```
        if (cin.fail())
        {
            cin.clear();
            cin.ignore(numeric_limits<streamsize>::max(), '\n');
            cout << "Invalid input! Please enter a number between 1 and 6.\n";
            continue;
        }

        switch (choice)
        {
            case 1:
            {
                cin.ignore();
                string text;
                cout << "Enter text to add: ";
                getline(cin, text);
                editor.addText(text);
                break;
            }
            case 2:
                editor.display();
                break;

            case 3:
                editor.undo();
                break;

            case 4:
                editor.redo();
                break;

            case 5:
                editor.clearText();
                break;

            case 6:
                cout << "Exiting program. Thank you!\n";
                break;
```

```
    do
        switch (choice)
            case 6:
                cout << "Exiting program. Thank you!\n";
                break;

            default:
                cout << "Invalid choice! Please select between 1 and 6.\n";
        }

    } while (choice != 6);

    return 0;
}
```

9. Sample Output

Below are sample runs of the program showing text entry, display, undo, redo, and exit. (Insert actual screenshots of your program's console output here when preparing the final submission.)

9.1 Adding and Displaying Text

```
Enter your choice (1-6): 1
Enter text to add: Hello
Text added successfully.

Enter your choice (1-6): 1
Enter text to add:  World
Text added successfully.

Enter your choice (1-6): 2
-----------------------------------
Current Text: Hello World
-----------------------------------
```

[Insert Screenshot 1: Adding text and displaying it]

9.2 Undo Operation

```
Enter your choice (1-6): 3
Undo successful.

Enter your choice (1-6): 2
-----------------------------------
Current Text: Hello
-----------------------------------
```

[Insert Screenshot 2: Undo operation in action]

9.3 Redo Operation

```
Enter your choice (1-6): 4
Redo successful.

Enter your choice (1-6): 2
-----------------------------------
Current Text: Hello World
-----------------------------------
```

[Insert Screenshot 3: Redo operation in action]

9.4 Clear Text and Exit

```
Enter your choice (1-6): 5
Text cleared.

Enter your choice (1-6): 6
Exiting program. Thank you!
```

[Insert Screenshot 4: Clearing text and exiting the program]

10. Advantages

- Simple and beginner-friendly demonstration of the Stack data structure.
- Clear separation between undo history and redo history using two stacks.
- Easy to read, modify, and extend for classroom demonstration.
- Runs on any system with a standard C++ compiler; no extra libraries needed.
- Helps in understanding LIFO logic through a relatable, real-life feature.

11. Limitations

- Works only in the console; there is no graphical user interface.
- Text is not saved to a file, so all data is lost when the program closes.
- Only whole-text snapshots are stored, not character-by-character or word-by-word changes.
- There is no limit control on stack size, so very large inputs could use more memory.
- Does not support multiple documents or advanced text formatting.

12. Applications

- Forms the core logic behind Undo/Redo features in word processors like MS Word or Notepad++.
- Similar stack-based logic is used in image editing software (undo a filter or brush stroke).
- Used in spreadsheet applications to undo/redo cell changes.
- Useful as a teaching example for browser back/forward navigation, which also uses stacks.
- Helpful base project for students learning STL containers such as std::stack.

13. Conclusion

This project successfully demonstrates how the Stack data structure can be used to implement the Undo and Redo features of a text editor. By maintaining two stacks - one for undo history and one for redo history - the program is able to move backward and forward through the changes made to a piece of text in a simple and predictable way.

Through this project, the concepts of Stack operations (push, pop, top), LIFO behaviour, and modular C++ programming using classes were applied practically. The project remains intentionally simple, in line with its purpose as a classroom-level academic exercise, and can be extended in the future with features such as file saving, character-level undo, or a graphical interface.

14. References

- Herbert Schildt, “C++: The Complete Reference”, Tata McGraw-Hill Education.
- E. Balagurusamy, “Object Oriented Programming with C++”, McGraw Hill Education.
- cplusplus.com - C++ Standard Library reference (std::stack, std::string).
- GeeksforGeeks.org - Articles on Stack data structure and its applications.
- cppreference.com - Official C++ Standard Library documentation.